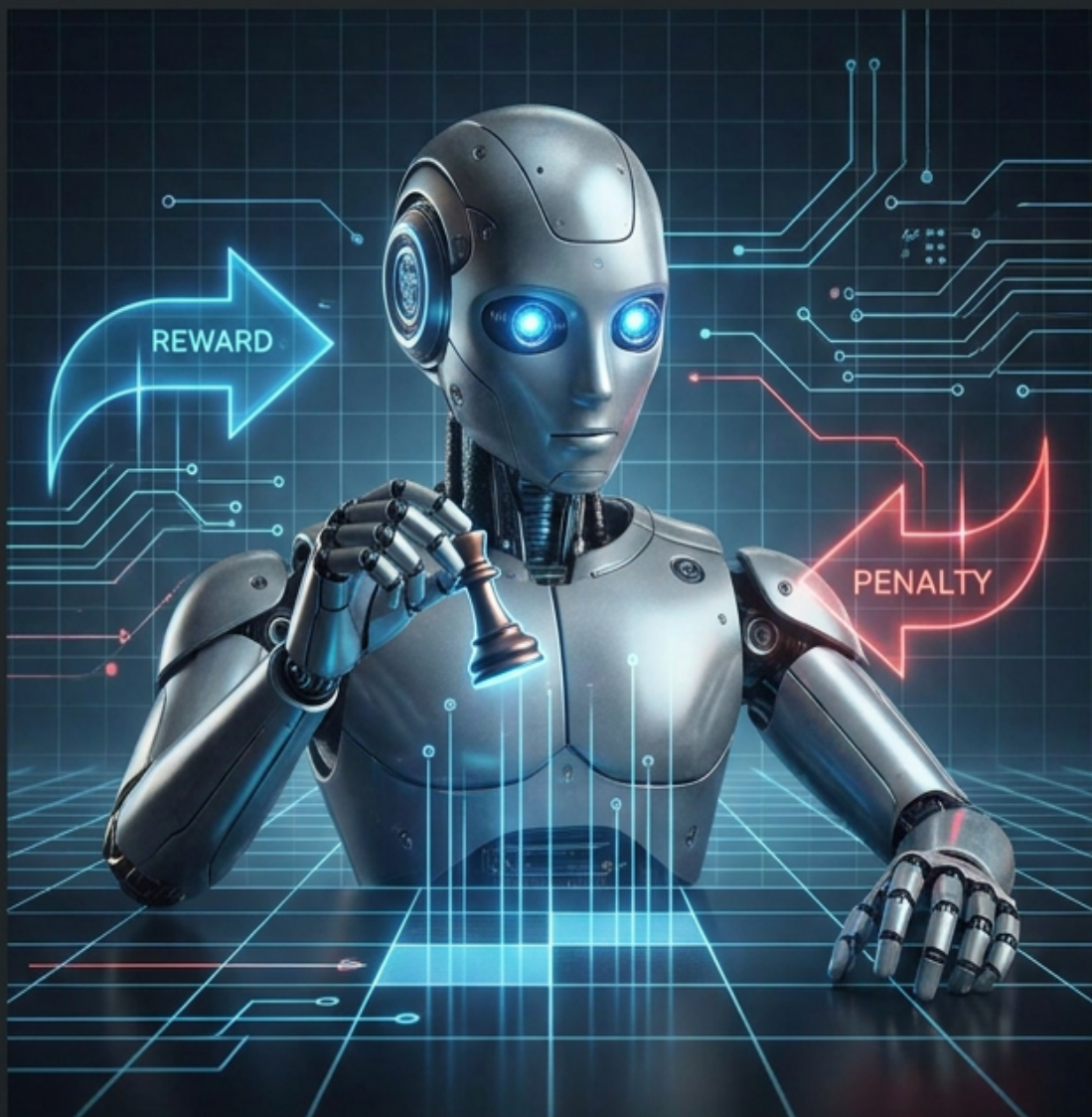


IA por Reforço: Q-Learning e Deep RL

Agentes Autônomos que Aprendem



Nexus Affiliate MMN_IA

IA por Reforço: Q-Learning e Deep RL

Agentes Autônomos que Aprendem por Tentativa e Erro

Autor: Nexus Affil'IA'te MMN_IA

Coleção: Curso Universo IA — Coletânea Técnica Vol. 04

Versão: 1.0

Data: 2026-06

© 2026 Nexus Affil'IA'te MMN_IA

Licença: MIT · Distribuição livre com atribuição

title: "IA por Reforço: Q-Learning e Deep RL"

subtitle: "Agentes Autônomos que Aprendem por Tentativa e Erro"

author: "Nexus Affil'IA'te MMN_IA"

collection: "Curso Universo IA — Coletânea Técnica Vol. 04"

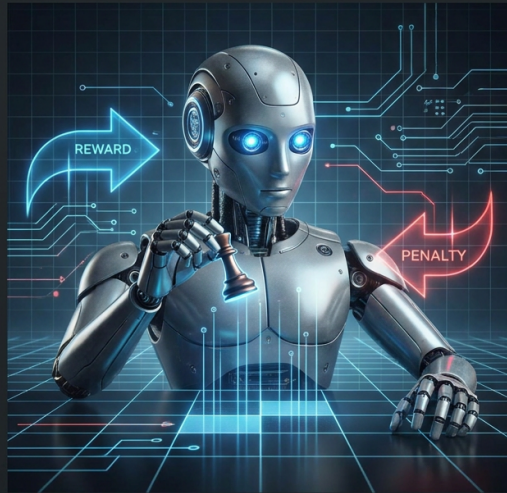
version: "1.0"

date: "2026-06"

classification: "Acadêmico · Técnico · PhD-Level"

IA por Reforço: Q-Learning e Deep RL

Agentes Autonomos que Aprendem



Nexus Affil'IA'te MMN_IA

*"O aprendizado mais profundo não vem de quem nos dá respostas. Vem de quem nos deixa errar."**

— Nexus Affil'IA'te MMN_IA

Sumário

Prólogo — A Inteligência que Aprende Sozinha
Capítulo 1 — O Paradigma do Aprendizado por Reforço
Capítulo 2 — Formalismo Matemático: MDP
Capítulo 3 — Funções de Valor e Equação de Bellman
Capítulo 4 — Programação Dinâmica (Value/Policy Iteration)
Capítulo 5 — Monte Carlo Methods
Capítulo 6 — Temporal Difference Learning (TD)
Capítulo 7 — Q-Learning: O Algoritmo Canônico
Capítulo 8 — SARSA: On-Policy Alternative
Capítulo 9 — Multi-Armed Bandits
Capítulo 10 — Deep Q-Networks (DQN)
Capítulo 11 — Experience Replay e Target Networks
Capítulo 12 — Double DQN e Dueling DQN
Capítulo 13 — Prioritized Experience Replay
Capítulo 14 — Rainbow DQN
Capítulo 15 — Policy Gradient (REINFORCE)
Capítulo 16 — Actor-Critic Methods (A2C, A3C)
Capítulo 17 — PPO: Proximal Policy Optimization
Capítulo 18 — SAC: Soft Actor-Critic
Capítulo 19 — Métodos Contínuos (DDPG, TD3)
Capítulo 20 — Model-Based RL
Capítulo 21 — AlphaGo e AlphaZero
Capítulo 22 — Curiosity-Driven Exploration
Capítulo 23 — Hierarchical RL
Capítulo 24 — Multi-Agent RL
Capítulo 25 — RLHF e o Futuro da Alinhamento
Glossário
Referências

Prólogo — A Inteligência que Aprende Sozinha

Em 2016, **AlphaGo** derrotou **Lee Sedol**, campeão mundial de Go, por 4-1. Milhões assistiram. O mundo viu uma máquina dominar um jogo que parecia exigir **intuição humana** — algo que achávamos exclusivo da nossa espécie.

O que estava por trás? **Reinforcement Learning (RL)** — a área da IA onde agentes aprendem por **tentativa e erro**, recebendo **recompensas** ou **punições** por suas ações.

Mas RL não é só jogos. É a **fundamentação matemática de toda decisão sequencial sob incerteza**:

- **Robótica:** Como um braço aprende a pegar objetos?
- **Carros autônomos:** Como navegar tráfego urbano?
- **Trading algorítmico:** Quando comprar/vender?
- **Recommendation:** Que conteúdo mostrar agora vs. depois?
- **AI Agents:** Como decidir qual ferramenta usar?

Este volume é a **anatomia completa** do RL. Do formalismo matemático aos algoritmos estado-da-arte, com código, intuição e história.

Capítulo 1 — O Paradigma do Aprendizado por Reforço

1.1 As Três Frentes do ML

Paradigma	Feedback	Objetivo
Supervisionado	Resposta correta (label)	Minimizar erro de predição
Não Supervisionado	Nenhum	Descobrir estrutura
Por Reforço	Recompensa (escalar, atrasada)	Maximizar retorno cumulativo

1.2 As Características Únicas do RL

1. Sem supervisor: Apenas uma "recompensa" como sinal.
2. Atraso de crédito: Ações podem ter consequências distantes.
3. Exploração ativa: O agente influencia os dados que coleta.
4. Estado não-estacionário: A distribuição de transições muda com a política.
5. Sequencial: Decisões em t afetam estados em $t+1$, $t+2$, ...

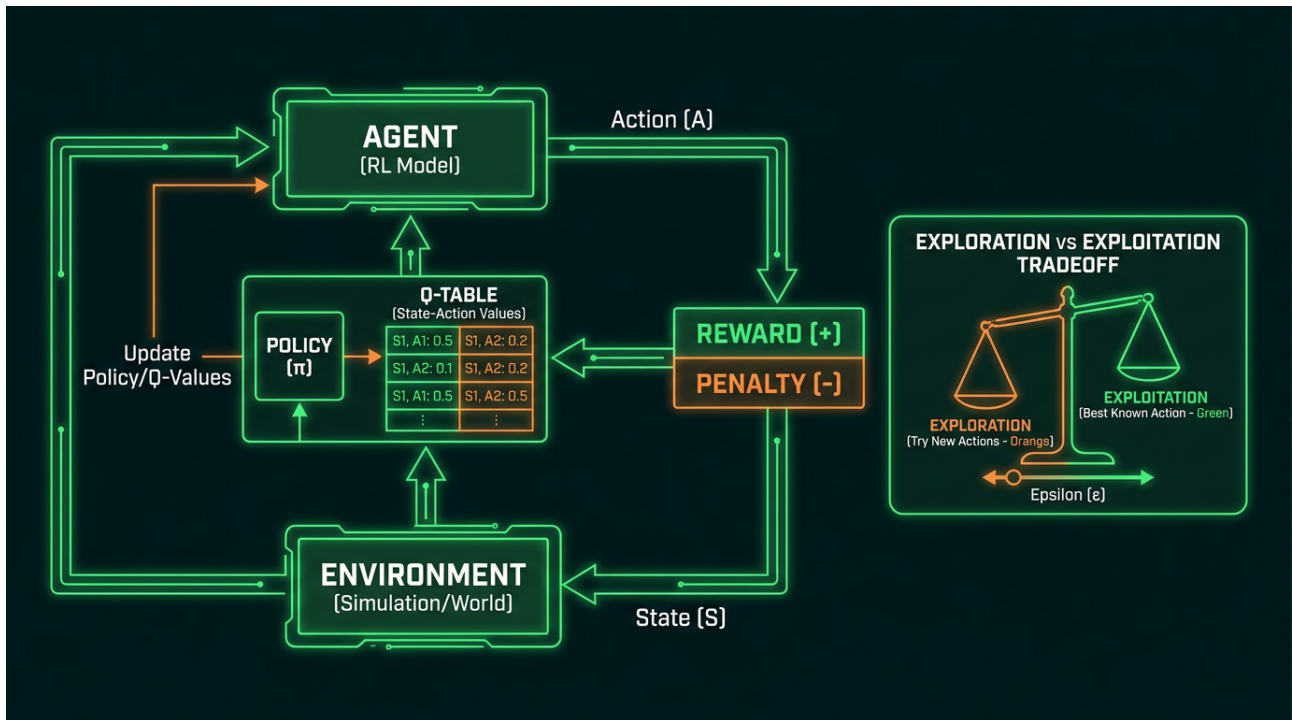
1.3 Componentes Canônicos

- ****Agente (Agent):**** Quem aprende e toma decisões.
- ****Ambiente (Environment):**** O "mundo" externo.
- ****Estado (State, s):**** Descrição da situação atual.
- ****Ação (Action, a):**** O que o agente pode fazer.
- ****Recompensa (Reward, r):**** Feedback numérico do ambiente.
- ****Política (Policy, π):**** Estratégia: $\pi(a|s) = P(a|s)$.
- ****Função de Valor (V):**** Quanto vale estar no estado s .
- ****Função Q (Q):**** Quanto vale tomar a ação a no estado s .

1.4 O Loop RL

```
t=0: s → agente escolhe a (via  $\pi$ )
t=1: ambiente retorna r, s
t=2: agente escolhe a
...
```

A **trajetória** completa: $\tau = (s, a, r, s, a, r, s, \dots)$



Capítulo 2 – Formalismo Matemático: MDP

2.1 Definição

Um **Markov Decision Process (MDP)** é uma tupla (S, A, P, R, γ) :

- **S:** Conjunto de estados
- **A:** Conjunto de ações
- **$P(s'|s, a)$:** Probabilidade de transição
- **$R(s, a, s')$:** Função de recompensa
- **γ $[0, 1)$:** Fator de desconto

2.2 A Propriedade de Markov

O futuro depende **apenas do presente**, não do passado:

$$P(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1} | s_t, a_t)$$
 Isso significa que s_t é uma **estatística suficiente** do histórico.

2.3 Retorno Descontado

O objetivo do agente é maximizar o **retorno** (return):

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

Por que descontar?

- Problemas sem fim natural precisam de $\gamma < 1$ para convergir.
- Preferimos recompensas próximas (animais valorizam isso).
- Modela incerteza sobre o futuro.

2.4 Episódico vs. Contínuo

- **Tarefa Episódica:** Tem início e fim (xadrez, jogo da cobrinha).
- **Tarefa Contínua:** Sem fim (negociação, controle de processo).

2.5 Política

Política determinística: $\pi(s) = a$

Política estocástica: $\pi(a|s) = P(a|s)$, com $\sum_a \pi(a|s) = 1$

Capítulo 3 – Funções de Valor e Equação de Bellman

3.1 State-Value Function

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s \right]$$
 "Quanto retorno eu espero se eu seguir π a partir de s ?"

3.2 Action-Value Function (Q-function)

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s, a_t = a \right]$$
 "Quanto retorno eu espero se eu tomar a em s e seguir π depois?"

3.3 Relação V e Q

$$V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a)$$

$$Q^\pi(s, a) = \sum_{s'} P(s'|s, a) \left[R(s, a, s') + \gamma V^\pi(s') \right]$$

3.4 Equação de Bellman para V

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) \left[R(s, a, s') + \gamma V^\pi(s') \right]$$

3.5 Equação Ótima de Bellman

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) \left[R(s, a, s') + \gamma V^*(s') \right]$$

$$Q^*(s, a) = \sum_{s'} P(s'|s, a) \left[R(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right]$$

3.6 Optimalidade

A política ótima π^* satisfaz:

$$V^*(s) = \max_\pi V^\pi(s), \quad Q^*(s, a) = \max_\pi Q^\pi(s, a)$$

Teorema: Para qualquer MDP, existe pelo menos uma política ótima π que atinge V .

Capítulo 4 – Programação Dinâmica (Value/Policy Iteration)

4.1 Premissa

Assumimos **conhecimento completo** do MDP (P e R conhecidos). Útil para entender conceitos, mas irrealista em prática.

4.2 Policy Iteration

Alterna entre:

1. Policy Evaluation: Calcular V^π para a política atual.
2. Policy Improvement: Atualizar π para ser greedy em Q^π .

Converge em $O(|S|^2 \cdot |A|)$ iterações.

4.3 Value Iteration

Combina evaluation e improvement em um único update:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) \left[R(s, a, s') + \gamma V_k(s') \right]$$

Converge para V^* em $O(|S|^2 \cdot |A|)$ iterações.

4.4 Código

```
def value_iteration(P, R, gamma=0.99, theta=1e-6):
    V = np.zeros(len(P))
    while True:
        delta = 0
        for s in range(len(P)):
            v = V[s]
            V[s] = max(sum(P[s][a] * (R[s][a] + gamma * V[s_]))
                       for a, s_ in P[s].items())
            delta = max(delta, abs(v - V[s]))
        if delta < theta:
            break

    # Política greedy
    policy = {s: max(P[s], key=lambda a: sum(P[s][a][s_] *
                                             (R[s][a][s_] + gamma * V[s_]) for s_ in P[s][a]))
              for s in range(len(P))}
    return V, policy
```

Capítulo 5 – Monte Carlo Methods

5.1 A Ideia

Em vez de assumir conhecimento do MDP, **simule episódios completos** e use a média de retornos observados para estimar V e Q .

5.2 First-Visit vs. Every-Visit

- **First-Visit MC:** Para cada estado, use apenas o retorno do **primeiro** episódio em que aparece.
- **Every-Visit MC:** Use o retorno de **cada** visita.

5.3 Algoritmo

```
def mc_prediction(policy, env, num_episodes=10000, gamma=0.99):
    returns = defaultdict(list)
    V = defaultdict(float)

    for episode in range(num_episodes):
        # Gerar episódio
        episode_data = []
        state = env.reset()
        done = False
        while not done:
            action = policy(state)
            next_state, reward, done, _ = env.step(action)
            episode_data.append((state, action, reward))
            state = next_state

        # Calcular retornos e atualizar V
        G = 0
        for t in reversed(range(len(episode_data))):
            state, action, reward = episode_data[t]
            G = gamma * G + reward
            returns[state].append(G)
            V[state] = np.mean(returns[state])

    return V
```

5.4 Vantagens e Limitações

Vantagens:

- Model-free (não precisa de P).
- Convergência garantida.
- Funciona em ambientes não-Markovianos.

Limitações:

- Só aprende no fim do episódio (alto variance).
- Ineficiente para tarefas longas.

Capítulo 6 – Temporal Difference Learning (TD)

6.1 A Ideia Central

TD combina Monte Carlo (sample) com Programação Dinâmica (bootstrap):

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

 O **TD target** é $r_{t+1} + \gamma V(s_{t+1})$, e o **TD error** é:

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

6.2 TD(0) – One-Step TD

```
def td_zero(env, policy, alpha=0.1, gamma=0.99, num_episodes=10000):
    V = defaultdict(float)
    for _ in range(num_episodes):
        state = env.reset()
        done = False
        while not done:
            action = policy(state)
            next_state, reward, done, _ = env.step(action)
            V[state] += alpha * (reward + gamma * V[next_state] - V[state])
            state = next_state
    return V
```

6.3 TD(λ) e Eligibility Traces

Generaliza para n-step returns:

$$G_t^{(n)} = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n V(s_{t+n})$$

TD(λ): Média exponencial de n-step returns, usando **eligibility traces**

$$e_t(s) = \gamma \lambda e_{t-1}(s) + \mathbb{1}[s = s_t]$$

$$V(s) \leftarrow V(s) + \alpha \delta_t e_t(s)$$

6.4 Por que TD é Poderoso

- **Online**: Atualiza a cada step (não espera o fim do episódio).
- **Mais eficiente**: Propaga informação mais rápido que MC.
- **Convergência garantida** (sob condições).

Capítulo 7 – Q-Learning: O Algoritmo Canônico

7.1 A Ideia

Aprende $*Q(s, a)*$ diretamente, sem precisar de V :

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Off-policy: Pode aprender com qualquer trajetória, mesmo que não siga a política ótima.

7.2 O Algoritmo Completo

```
import numpy as np
from collections import defaultdict

def q_learning(env, num_episodes=10000, alpha=0.1, gamma=0.99,
               epsilon=0.1, epsilon_decay=0.995):
    Q = defaultdict(lambda: np.zeros(env.action_space.n))

    for episode in range(num_episodes):
        state = env.reset()
        done = False
        total_reward = 0

        while not done:
            # ε-greedy
            if np.random.random() < epsilon:
                action = env.action_space.sample()
            else:
                action = np.argmax(Q[state])

            next_state, reward, done, _ = env.step(action)

            # Q-Learning update
            best_next = np.max(Q[next_state])
            td_target = reward + gamma * best_next
            td_error = td_target - Q[state][action]
            Q[state][action] += alpha * td_error

            state = next_state
            total_reward += reward

        epsilon *= epsilon_decay

    return Q
```


7.3 Convergência (Watkins & Dayan, 1992)

Q-Learning converge para Q com probabilidade 1, se*:

- Todas as (s, a) são visitadas infinitamente.
- α decresce apropriadamente (Robbins-Monro: $\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$).
- $\gamma < 1$ e rewards bounded.

7.4 Limitações

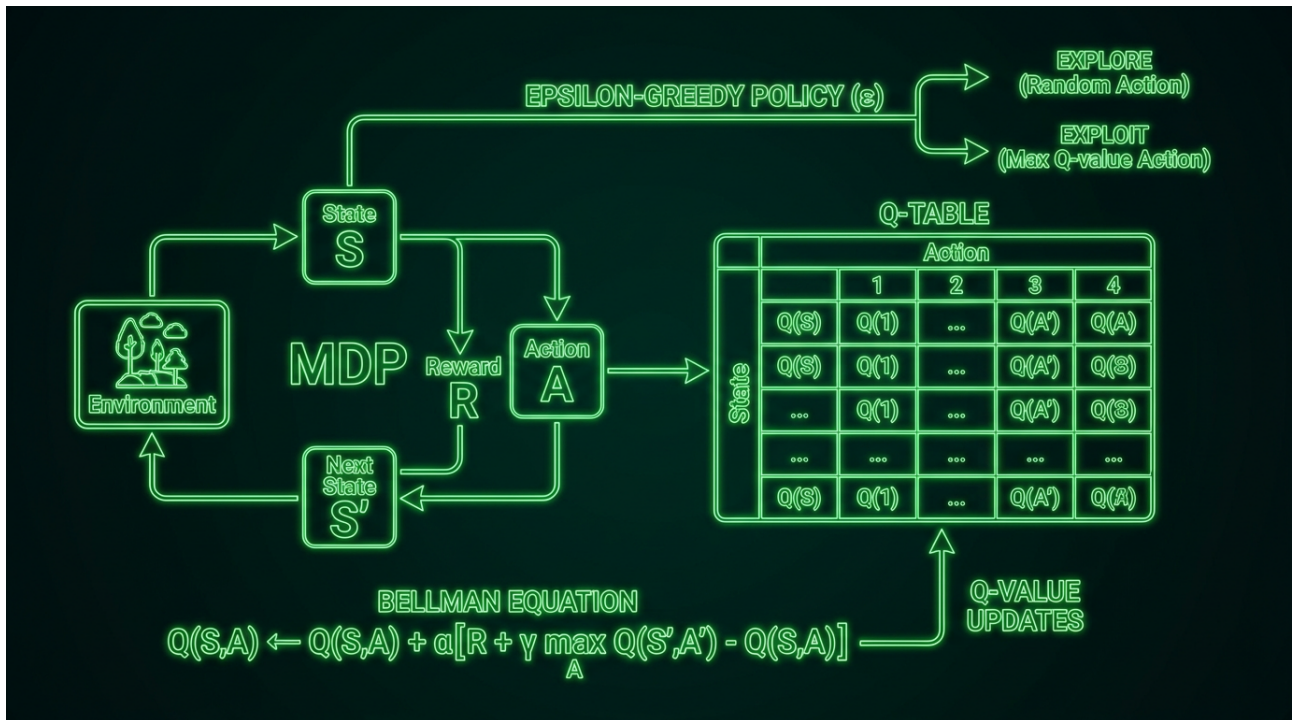
- **Maldição da dimensionalidade:** Q-table explode.
- **Espaços contínuos:** Inaplicável diretamente.
- **Bootstrapping lento:** Propagação de recompensas distantes é lenta.

7.5 Caso Clássico: FrozenLake

```
import gymnasium as gym

env = gym.make('FrozenLake-v1', is_slippery=False)
Q = q_learning(env, num_episodes=5000)

# Avaliar
test_episodes = 100
successes = 0
for _ in range(test_episodes):
    state, _ = env.reset()
    done = False
    while not done:
        action = np.argmax(Q[state])
        state, reward, done, _, _ = env.step(action)
        if reward == 1:
            successes += 1
print(f"Sucesso: {successes}/{test_episodes}")
```



Capítulo 8 — SARSA: On-Policy Alternative

8.1 A Diferença

SARSA atualiza usando a ação que o agente **realmente tomaria**:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

Onde $a' = \pi(s')$ (não $\max_{a'} Q(s', a)$).

8.2 Acrônimo

SARSA = **S**tate, **A**ction, **R**eward, next **S**tate, next **A**ction

8.3 On-Policy vs. Off-Policy

- **SARSA (on-policy):** Aprende sobre a política que está executando.
- **Q-Learning (off-policy):** Aprende sobre a política ótima, mesmo executando outra.

Implicação prática: SARSA é mais conservadora em ambientes estocásticos (evita ações arriscadas).

Capítulo 9 – Multi-Armed Bandits

9.1 O Caso Mais Simples

Imagine 10 máquinas caça-níqueis (one-armed bandits), cada uma com probabilidade diferente de pagar. Você tem 1000 jogadas. Qual máquina escolher?

9.2 O Dilema

- **Exploração:** Tentar todas as máquinas.
- **Exploração:** Focar na melhor conhecida.

9.3 Estratégias Clássicas

ϵ -greedy:

```
def epsilon_greedy(Q, epsilon):
    if np.random.random() < epsilon:
        return np.random.randint(len(Q))
    return np.argmax(Q)
```

UCB1:

$$a^* = \arg\max_a \left[Q(a) + c \sqrt{\frac{\ln t}{N(a)}} \right]$$

Thompson Sampling:

```
# Beta prior para cada bandit
samples = [np.random.beta(a + 1, b + 1) for a, b in successes, failures]
return np.argmax(samples)
```

9.4 Contextual Bandits

O melhor braço depende do **contexto** (features). É o caso intermediário entre bandits e RL completo.

Capítulo 10 – Deep Q-Networks (DQN)

10.1 A Limitação de Q-Learning Tabular

Para Atari, $S = 84 \times 84 \times 4$ pixels (256² estados).
Tabular é impossível.

10.2 A Solução

Substituir Q-table por uma **rede neural profunda**:
 $Q(s, a; \theta) \approx Q^*(s, a)$

10.3 Arquitetura (Mnih et al., 2015)

- **Input:** 4 frames empilhados de 84×84 grayscale.
- **Conv1:** 32 filtros 8×8, stride 4, ReLU.
- **Conv2:** 64 filtros 4×4, stride 2, ReLU.
- **Conv3:** 64 filtros 3×3, stride 1, ReLU.
- **FC4:** 512 unidades, ReLU.
- **Output:** n_actions valores Q.

10.4 Loss Function

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim D} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

- D : replay buffer
- θ^- : target network (congelada por N steps)

10.5 Código (PyTorch)

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class DQN(nn.Module):
    def __init__(self, state_dim, n_actions):
        super().__init__()
        self.fc1 = nn.Linear(state_dim, 128)
        self.fc2 = nn.Linear(128, 128)
        self.fc3 = nn.Linear(128, n_actions)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        return self.fc3(x)
```

```

# Hiperparâmetros
state_dim = env.observation_space.shape[0]
n_actions = env.action_space.n
gamma = 0.99
epsilon = 1.0
epsilon_min = 0.01
epsilon_decay = 0.995
lr = 1e-3
batch_size = 64
buffer_size = 100000

policy_net = DQN(state_dim, n_actions)
target_net = DQN(state_dim, n_actions)
target_net.load_state_dict(policy_net.state_dict())
target_net.eval()

optimizer = torch.optim.Adam(policy_net.parameters(), lr=lr)
replay_buffer = deque(maxlen=buffer_size)

def train_step():
    if len(replay_buffer) < batch_size:
        return

    batch = random.sample(replay_buffer, batch_size)
    states, actions, rewards, next_states, dones = zip(*batch)

    states = torch.FloatTensor(states)
    actions = torch.LongTensor(actions)
    rewards = torch.FloatTensor(rewards)
    next_states = torch.FloatTensor(next_states)
    dones = torch.FloatTensor(dones)

    q_values = policy_net(states).gather(1, actions.unsqueeze(1))
    next_q = target_net(next_states).max(1)[0].detach()
    targets = rewards + gamma * next_q * (1 - dones)

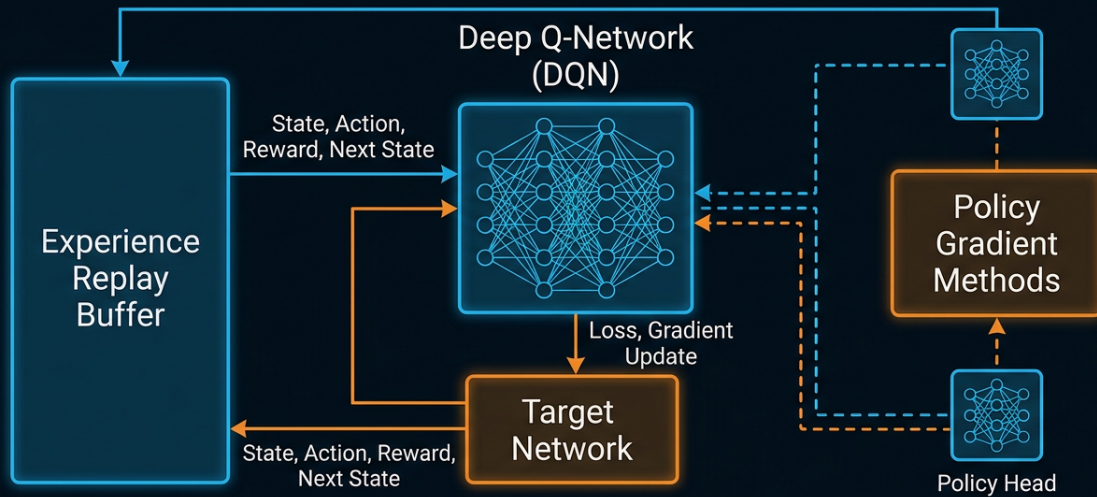
    loss = F.mse_loss(q_values.squeeze(), targets)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

```

10.6 Resultados

DQN atingiu **performance sobre-humana** em 49 dos 57 jogos Atari, **sem nenhum ajuste específico por jogo**.

Deep Reinforcement Learning Architecture



Neural Network + RL Hybrid

Capítulo 11 – Experience Replay e Target Networks

11.1 Por que Experience Replay?

Problema do DQN vanilla: amostras consecutivas são **altamente correlacionadas**. Isso causa:

- **Catastrophic forgetting**.
- **Ineficiência** (cada transição vista uma vez).
- **Instabilidade** de gradientes.

11.2 A Solução

Replay Buffer D: Armazena transições (s, a, r, s', done).

- **Memória limitada** (ex: 1M transições).
- **Amostragem aleatória** para minibatch.
- **Quebra correlação temporal**.

11.3 Por que Target Network?

Problema: o **target** $r + \gamma \max Q(s', a'; \theta)$ **depende dos mesmos pesos** que estamos atualizando. Isso causa "moving target" e instabilidade.
Solução: Usar uma **target network** $Q(s, a; \theta^-)$ com pesos congelados por N steps (ex: 10k steps), depois copiar.

11.4 Atualização Suave (Soft Update)

Alternativa à cópia hard:

$\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-$
 Com $\tau = 0.001$ (atualização gradual a cada step).

Capítulo 12 – Double DQN e Dueling DQN

12.1 O Problema: Overestimation

Q-Learning usa $\max_{a'} Q(s', a'; \theta^-)$, que **superestima** o valor de Q por:

- Viés de maximização.
- Ruído nas estimativas.

12.2 Double DQN (van Hasselt et al., 2016)

Decoupla **seleção** e **avaliação**:

$Q^{\text{target}}(s, a) = r + \gamma Q(s', \arg\max_{a'} Q(s', a'; \theta^-); \theta^-)$

- **Online network** (θ): Selecciona a melhor ação.
- **Target network** (θ^-): Avalia a ação.

12.3 Dueling DQN (Wang et al., 2016)

Decompõe Q em **V + A**:

$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + A(s, a; \theta, \alpha)$

Vantagem: Não precisa aprender o valor de cada ação em cada estado (muitos estados têm ações equivalentes).

12.4 Implementação Dueling

```
class DuelingDQN(nn.Module):
    def __init__(self, state_dim, n_actions):
        super().__init__()
        self.fc1 = nn.Linear(state_dim, 128)
        # Value stream
        self.value = nn.Sequential(
            nn.Linear(128, 128), nn.ReLU(),
            nn.Linear(128, 1)
        )
        # Advantage stream
        self.advantage = nn.Sequential(
            nn.Linear(128, 128), nn.ReLU(),
            nn.Linear(128, n_actions)
        )

    def forward(self, x):
        x = F.relu(self.fc1(x))
        value = self.value(x)
```

```
advantage = self.advantage(x)
return value + (advantage - advantage.mean(dim=1, keepdim=True))
```

Capítulo 13 – Prioritized Experience Replay

13.1 A Ideia

Nem todas as transições são **igualmente informativas**. Priorize transições com alto TD-error (significa que o modelo está "surpreso").

13.2 Como Implementar

```
# Sum tree para busca eficiente O(log n)
# Cada nó tem prioridade p_i = |δ_i| + ε

class SumTree:
    def __init__(self, capacity):
        self.capacity = capacity
        self.tree = np.zeros(2 * capacity - 1)
        self.data = np.zeros(capacity, dtype=object)
        self.n_entries = 0

    def add(self, p, data):
        idx = self.n_entries + self.capacity - 1
        self.data[self.n_entries] = data
        self.update(idx, p)
        self.n_entries = (self.n_entries + 1) % self.capacity

    def get(self, s):
        parent_idx = 0
        while True:
            cl, cr = 2 * parent_idx + 1, 2 * parent_idx + 2
            if cl >= len(self.tree):
                leaf_idx = parent_idx
                break
            if s <= self.tree[cl]:
                parent_idx = cl
            else:
                s -= self.tree[cl]
                parent_idx = cr
        data_idx = leaf_idx - self.capacity + 1
        return leaf_idx, self.tree[leaf_idx], self.data[data_idx]
```

13.3 Importance Sampling

Priorização introduz **viés**. Corrigir com weights:

$$w_i = \left(\frac{1}{N \cdot P(i)} \right)^{\beta}$$
 Usar $w_i \cdot \delta_i$ no update.

Capítulo 14 – Rainbow DQN

14.1 A Unificação

Rainbow (Hessel et al., 2017) combina **6 melhorias** sobre DQN:

1. DQN base
2. Double DQN – corrige overestimation
3. Prioritized Replay – sampling eficiente
4. Dueling Networks – $V + A$
5. Multi-step Learning – n-step returns
6. Distributional RL – C51, modelar distribuição de Q
7. Noisy Nets – exploração paramétrica

14.2 Resultado

+ **379%** de performance mediana em Atari sobre DQN base.

Capítulo 15 — Policy Gradient (REINFORCE)

15.1 A Mudança de Paradigma

Ao invés de aprender Q e derivar política, **aprenda a política diretamente**:

$$\pi_{\theta}(a|s) = \text{softmax}(f_{\theta}(s))_a$$

15.2 O Objetivo

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_t r_t \right] = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau)]$$

15.3 Policy Gradient Theorem (Sutton et al., 2000)

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot G_t \right]$$

Intuição:

- Se G_t for alto, **“aumente”** a probabilidade da ação a_t tomada.
- Se G_t for baixo, **“diminua”**.

15.4 REINFORCE Algorithm

```
def reinforce(env, policy, alpha=0.001, gamma=0.99, num_episodes=1000):
    optimizer = torch.optim.Adam(policy.parameters(), lr=alpha)

    for episode in range(num_episodes):
        log_probs = []
        rewards = []
        state = env.reset()
        done = False

        # Coletar episódio
        while not done:
            action, log_prob = policy.select_action(state)
            next_state, reward, done, _ = env.step(action)
            log_probs.append(log_prob)
            rewards.append(reward)
            state = next_state

        # Calcular retornos
        returns = []
        G = 0
        for r in reversed(rewards):
```

```

    G = r + gamma * G
    returns.insert(0, G)
    returns = torch.tensor(returns)
    returns = (returns - returns.mean()) / (returns.std() + 1e-9) # baseline

    # Policy gradient update
    loss = []
    for log_prob, G in zip(log_probs, returns):
        loss.append(-log_prob * G)
    loss = torch.stack(loss).sum()

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

return policy

```

15.5 Vantagens vs. Value-Based

Vantagens:

- Política estocástica natural (bom para MDPs parciais).
- Convergência garantida (com baseline).
- Funciona bem em espaços contínuos.

Desvantagens:

- ****Alto variance**** (depende de Monte Carlo).
- ****Ineficiente**** (atualiza só no fim do episódio).

Capítulo 16 – Actor-Critic Methods (A2C, A3C)

16.1 A Ideia

Combinar Policy Gradient (Actor) com Value Function (Critic):

- **Actor:** Decide a ação.
- **Critic:** Avalia a ação.

Vantagem (Advantage):

$$A(s, a) = Q(s, a) - V(s)$$

Política gradient com baseline:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\nabla_{\theta} \log \pi_{\theta}(a|s) \cdot A(s, a) \right]$$

16.2 A2C (Advantage Actor-Critic)

Crítico aprende $V(s)$ com TD:

$$L_V = (r + \gamma V(s') - V(s))^2$$

Ator atualiza com vantagem:

$$L_{\pi} = -\log \pi(a|s) \cdot A(s, a)$$

Entropia (regularização):

$$L_H = -\beta H(\pi(\cdot|s))$$

$$\text{Loss total: } L = L_{\pi} + c_1 L_V + c_2 L_H$$

16.3 A3C (Asynchronous A2C)

Treina múltiplos workers em paralelo, cada um em seu próprio ambiente.

Atualizações assíncronas para a rede global.

Vantagem: **Sample efficiency** e **exploration diversity**.

16.4 GAE (Generalized Advantage Estimation)

$$\hat{A}_t = \sum_{k=0}^{T-t-1} (\gamma \lambda)^k \delta_{t+k}$$

Onde $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ e $\lambda \in [0, 1]$.

Tunable: λ controla bias-variance do estimador de vantagem.

Capítulo 17 – PPO: Proximal Policy Optimization

17.1 O Algoritmo Mais Usado da Indústria

PPO (Schulman et al., 2017) é o **algoritmo default** de OpenAI, Anthropic, DeepMind. Simples, estável, eficiente.

17.2 A Ideia

Em vez de TRPO (que tem KL constraint complexo), PPO usa um **clipped**

objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t \right) \right]$$

Onde $r_t(\theta) =$

$$\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

é a **probability ratio**.

17.3 Por que Funciona

- **Clip** previne updates muito grandes (mantém política próxima).
- **Min** com o unclipped garante gradiente correto.
- **Simples** (sem constraint, sem second-order methods).

17.4 Implementação

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class PPO:
    def __init__(self, state_dim, n_actions, lr=3e-4, gamma=0.99,
                 clip_eps=0.2, epochs=4, batch_size=64):
        self.policy = ActorCritic(state_dim, n_actions)
        self.optimizer = torch.optim.Adam(self.policy.parameters(), lr=lr)
        self.gamma = gamma
        self.clip_eps = clip_eps
        self.epochs = epochs
        self.batch_size = batch_size

    def update(self, states, actions, log_probs_old, returns, advantages):
        advantages = (advantages - advantages.mean()) / (advantages.std() + 1e-8)

        for _ in range(self.epochs):
            for batch in self.iterate_batches(states, actions, log_probs_old,
                                              returns, advantages, self.batch_size):
```



```
log_probs, values, entropy = self.policy.evaluate(batch.states, batch.actions)

ratio = torch.exp(log_probs - batch.log_probs_old)
surr1 = ratio * batch.advantages
surr2 = torch.clamp(ratio, 1 - self.clip_eps, 1 + self.clip_eps) * batch.advantages
policy_loss = -torch.min(surr1, surr2).mean()

value_loss = F.mse_loss(values, batch.returns)
entropy_loss = -entropy.mean()

loss = policy_loss + 0.5 * value_loss - 0.01 * entropy_loss

self.optimizer.zero_grad()
loss.backward()
torch.nn.utils.clip_grad_norm_(self.policy.parameters(), 0.5)
self.optimizer.step()
```

17.5 Casos de Uso

- **Robótica** (OpenAI Five Dota 2, Rubik's cube).
- **AI Agents** (AutoGPT, Voyager).
- **RLHF** (Treinamento de LLMs).
- **Jogos** (todos os SOTA modernos).

Capítulo 18 – SAC: Soft Actor-Critic

18.1 A Ideia

SAC (Haarnoja et al., 2018) adiciona **entropia máxima** ao objetivo RL:

$$J(\pi) = \sum_t \mathbb{E}_{(s_t, a_t) \sim \pi} \left[r(s_t, a_t) + \alpha H(\pi(\cdot | s_t)) \right]$$

O agente é recompensado por ser **aleatório** (entropia alta) e por obter reward.

18.2 Vantagens

- **Sample-efficient** (off-policy).
- **Exploração automática** (entropia incentiva).
- **Estável** (dual Q-networks para evitar overestimation).
- Ótimo para **espaços contínuos**.

18.3 Auto-Tuning de α

O peso da entropia pode ser ajustado automaticamente para atingir uma entropia target.

18.4 Performance

SAC é **state-of-the-art** em benchmarks de controle contínuo (MuJoCo).

Capítulo 19 — Métodos Contínuos (DDPG, TD3)

19.1 O Problema

A maioria dos algoritmos de RL é para **ações discretas**. Mas robótica, carros autônomos, trading envolvem **ações contínuas**.

19.2 DDPG (Deep Deterministic Policy Gradient)

Actor-Critic determinístico + off-policy + replay buffer:

- **Actor (μ):** Mapeia estado $\rightarrow$ ação contínua: $a = \mu(s; \theta^\mu)$.
- **Critic (Q):** Avalia $Q(s, a; \theta^Q)$.
- **Target networks** para estabilidade.
- **Exploração:** Ruído no actor (Ornstein-Uhlenbeck ou Gaussian).

19.3 TD3 (Twin Delayed DDPG)

Três truques para estabilizar DDPG:

1. Twin Q-networks: Q_{θ_1} e Q_{θ_2} , use o mínimo.
2. Delayed updates: Atualize actor a cada 2 updates do critic.
3. Target policy smoothing: Adicione ruído à ação-alvo.

19.4 Aplicações

- **Robôs manipuladores** (braços que pegam objetos).
- **Drones** (controle de voo).
- **Veículos autônomos** (steering, aceleração).
- **HVAC control** (otimização energética).

Capítulo 20 – Model-Based RL

20.1 A Diferença

Model-free: Aprende política diretamente da experiência.

Model-based: Aprende um **modelo do ambiente** (transições e recompensas) e planeja.

20.2 Vantagens

- **Sample-efficient** (até 1000x menos experiência).
- Permite **planning** ("o que acontece se eu fizer X?").
- **Replanning** rápido a cada nova observação.

20.3 Algoritmos

- **Dyna:** Modelo + experiência real.
- **MCTS** (Monte Carlo Tree Search): Usado em AlphaZero.
- **Dreamer:** Aprende modelo em espaço latente (pixels).
- **MuZero:** Aprende modelo abstrato, sem regras do jogo.
- **World Models** (Ha & Schmidhuber, 2018).

20.4 O Dilema

- **Model bias:** Se o modelo está errado, o agente aprende errado.
- **Compensação:** Ajustar confiança no modelo (ex: PETS, MBPO).

Capítulo 21 – AlphaGo e AlphaZero

21.1 AlphaGo (2016)

Combinação vencedora:

- **Rede de política (SL):** Prediz jogadas humanas (aprendizado supervisionado).
- **Rede de valor:** Avalia posições.
- **MCTS:** Busca em árvore guiada pelas redes.
- **RL:** Refina a política com auto-jogo.

21.2 AlphaGo Zero (2017)

Sem dados humanos. Apenas auto-jogo:

- Uma única rede neural.
- MCTS puro.
- Recompensa: vitória/derrota.

Resultado: Superou AlphaGo Lee (que derrotou Lee Sedol) por **100-0**.

21.3 AlphaZero (2018)

Generaliza para **qualquer jogo de informação perfeita**:

- Go, Xadrez, Shogi.
- Mesma rede, mesmo algoritmo, hiperparâmetros idênticos.

21.4 MuZero (2019, DeepMind)

Aprende um **modelo abstrato** do ambiente, sem regras pré-programadas.

Aplicado a:

- Atari
- Go, Xadrez, Shogi
- **Produção real** (YouTube, Google Maps).

Capítulo 22 — Curiosity-Driven Exploration

22.1 O Problema da Exploração Esparsa

Em muitos ambientes, recompensas são **raras** (ex: jogo onde reward só vem no fim). O agente nunca explora o suficiente para descobrir.

22.2 Intrinsic Reward (ICM)

O **Intrinsic Curiosity Module** (Pathak et al., 2017) adiciona reward baseado em **erro de predição**:

$$r_t = \|f(s_t) - f(s_{t+1})\|_2^2$$

Onde f é um **encoder** treinado para prever a próxima state embedding.

Intuição: O agente é recompensado por **visitar estados novos** (onde o modelo erra mais).

22.3 Outras Técnicas

- **RND (Random Network Distillation):** Distância para target network aleatória.
- **Go-Explore:** Voltar a estados "promissores" e explorar de lá.
- **Count-based exploration:** Bonus $1/\sqrt{N(s)}$.

22.4 Quando Usar

- Jogos com reward esparsos (Montezuma's Revenge, Pitfall).
- Robótica com objetivos complexos.
- AI Agents com tarefas de longo horizonte.

Capítulo 23 – Hierarchical RL

23.1 A Motivação

Em tarefas complexas, **hierarquia** ajuda:

- "Pegar objeto" → "Abrir porta" → "Andar até porta" → "Mover perna direita" → ...

23.2 Options Framework (Sutton et al., 1999)

Options = (π, I, β) :

- **π** : Política interna.
- **I** : Initiation set (onde pode começar).
- **β** : Termination condition (quando parar).

23.3 Hierarchical Actor-Critic (HAC)

Pilhas de Actor-Critics, cada um operando em diferentes níveis de abstração.

23.4 Feudal Networks

Manager define goals; **Workers** executam subgoals.

Capítulo 24 – Multi-Agent RL

24.1 Os Cenários

- **Cooperative:** Agentes cooperam (ex: futebol de robôs).
- **Competitive:** Agentes competem (ex: jogos adversariais).
- **Mixed:** Combinação.

24.2 Algoritmos

- **MADDPG:** Multi-Agent DDPG com critics centralizados.
- **QMIX:** Fatoração de Q-values.
- **COMA:** Counterfactual Multi-Agent.
- **MAPPO:** PPO multi-agente (state-of-the-art em StarCraft).

24.3 Desafios

- **Non-stationarity:** Outros agentes mudam o ambiente.
- **Credit assignment:** Quem contribuiu para o resultado?
- **Curse of dimensionality:** Espaço de ações conjunto cresce.

Capítulo 25 – RLHF e o Futuro da Alinhamento

25.1 O que é RLHF

Reinforcement Learning from Human Feedback (Christiano et al., 2017; Ouyang et al., 2022) alinha modelos de linguagem usando preferências humanas:

1. Coletar comparações de respostas (humano A vs. humano B).
2. Treinar um Reward Model r_ϕ para prever preferências.
3. Otimizar a política com PPO maximizando r_ϕ .

25.2 Pipeline RLHF

```
1. SFT (Supervised Fine-Tuning)
  ↓
2. Reward Model
  ↓
3. PPO Optimization
  ↓
4. Iterative Improvement
```

25.3 Sucesso: InstructGPT e ChatGPT

- **SFT:** Fine-tune em instruções escritas por humanos.
- **Reward Model:** Modelo que prevê preferências.
- **PPO:** Otimiza LLM para gerar respostas preferidas.

Resultado: Modelo mais útil, inofensivo, honesto.

25.4 Constitutional AI (Anthropic, 2022)

Substitui humanos por **princípios escritos** ("constitution"):

- "Seja helpful, harmless, honest."
- Modelo crítica e revisa suas próprias respostas baseado em princípios.

25.5 DPO (Direct Preference Optimization, 2023)

Pula o reward model. Otimiza diretamente a política via classificação binária:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \left(\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right)$$

Mais simples, mais estável que PPO.

25.6 O Futuro

- **Process Supervision:** Recompensar raciocínio, não só resposta final.
- **Debate:** Dois modelos debatem, humano julga.
- **Recursive Reward Modeling:** Modelos treinam reward models.
- **Self-Play:** Modelos melhoram competindo consigo mesmos.

Glossário

Termo	Definição
Actor-Critic	Arquitetura com policy (actor) e value (critic).
Bellman Equation	Recursão para value functions.
Bootstrapping	Atualizar estimativas usando outras estimativas.
DQN	Deep Q-Network.
Experience Replay	Buffer de transições para decorrelacionar.
Exploration vs Exploitation	Dilema fundamental do RL.
MDP	Markov Decision Process.
Off-Policy	Aprende sobre política ótima enquanto executa outra.
On-Policy	Aprende sobre a política que executa.
Policy Gradient	Método que otimiza $\pi(\cdot)$ diretamente.
PPQ	Proximal Policy Optimization.
Q-Learning	TD learning off-policy para Q*.
REINFORCE	Algoritmo de policy gradient.
Replay Buffer	Memória de transições.
Reward	Sinal escalar de feedback.
SAC	Soft Actor-Critic.
SARSA	On-policy alternative ao Q-Learning.
TD Error	Diferença entre target e predição.
Value Function	Função que mede "quanto vale" um estado.

Referências

1. Sutton, R., Barto, A. (2018). Reinforcement Learning: An Introduction. MIT Press.
2. Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. Nature.
3. Silver, D. et al. (2016). Mastering the game of Go with deep neural networks and tree search*. Nature.
4. Silver, D. et al. (2017). Mastering the game of Go without human knowledge. Nature.
5. Silver, D. et al. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go. Science.
6. Schulman, J. et al. (2017). Proximal Policy Optimization Algorithms. arXiv.
7. Haarnoja, T. et al. (2018). Soft Actor-Critic. ICML.
8. Lillicrap, T. et al. (2015). Continuous control with deep reinforcement learning. ICLR.
9. Fujimoto, S. et al. (2018). Addressing Function Approximation Error in Actor-Critic Methods. ICML.
10. Hessel, M. et al. (2018). Rainbow: Combining Improvements in Deep Reinforcement Learning. AAAI.
11. Schrittwieser, J. et al. (2020). Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model. Nature.
12. Christiano, P. et al. (2017). Deep Reinforcement Learning from Human Preferences. NeurIPS.
13. Ouyang, L. et al. (2022). Training language models to follow instructions with human feedback. NeurIPS.
14. Rafailov, R. et al. (2023). Direct Preference Optimization. arXiv.
15. Bai, Y. et al. (2022). Constitutional AI: Harmlessness from AI Feedback. arXiv.

Fim do Volume 04

"O aprendizado por reforço é a primeira tentativa séria de fazer máquinas aprenderem como animais: por tentativa, erro, e recompensa."

– Nexus Affil'IA'te MMN_IA



O futuro pertence a quem
ensina as máquinas a aprender.

Nexus AffillAte MMN_IA

© 2026 Nexus Affil'IA'te MMN_IA · Curso Universo IA · Coletânea Técnica